Projekty jQuery – klasy 4

Każdy uczeń wybiera jeden projekt z poziomu odpowiadającego ocenie, o którą się stara (3, 4 lub 5). Projekty opierają się na materiale z zajęć w klasach 4 (Git oraz jQuery: selektory, zdarzenia, manipulacja DOM, klasy CSS, formularze i walidacja).

Zasady wykonania

- Technologie: HTML, CSS i jQuery (dołączone z CDN, jak na lekcjach). Dozwolone jest użycie Bootstrapa. Zabronione są frameworki JavaScript, takie jak React, Vue czy Angular.
- Cały kod JavaScript znajduje się wewnątrz `$(function(){ ... })`.
- Projekt jest prowadzony w repozytorium Git. Minimalna liczba commitów: 1 commit na rozpoczęcie projektu + 1 commit na każde wymaganie (dokładna liczba jest podana przy każdym projekcie). Każde wymaganie powinno trafić do repozytorium jako osobny commit. Opis commita mówi, co zostało zrobione, np. „Dodano walidację pola e-mail w formularzu" albo „Dodano licznik produktów w koszyku", a nie „poprawki", „zmiany" czy „asdf". Commity są rozłożone w czasie pracy, a nie wrzucone jednorazowo na koniec.
- Kod jest czytelny: wcięcia, nazwy zmiennych i identyfikatorów mówią, co przechowują, np. cenaKoszyka, liczbaProb, #przyciskDodaj, a nie a, x2 czy #div1; krótkie komentarze przy najważniejszych fragmentach.
- Strona ma estetyczny, spójny wygląd przygotowany w CSS (samodzielnie lub z użyciem Bootstrapa).
- Projekt musi działać bez błędów w konsoli przeglądarki.

Oddanie i obrona

- Uczeń oddaje link do repozytorium i prezentuje działający projekt.
- Podczas obrony uczeń musi wyjaśnić działanie dowolnego wskazanego fragmentu kodu oraz wprowadzić na miejscu niewielką zmianę (np. zmienić komunikat, warunek lub dodać prosty element).
- Projekt, w którym brakuje wymagań, może otrzymać ocenę niższą niż poziom, na który był wybrany. Ostateczna decyzja co do oceny należy do nauczyciela.
- **Jeżeli uczeń nie odda projektu do ostatecznego terminu zapisanego w pliku z zapisami, otrzymuje ocenę niedostateczną (1).**

Zasady dotyczące sztucznej inteligencji (AI)

- Korzystanie z narzędzi AI (ChatGPT, Claude, Copilot, Gemini itp.) bez zrozumienia otrzymanego kodu jest **zakazane**.
- Uczeń odpowiada za każdą linię kodu w projekcie i musi umieć ją wyjaśnić.
- **Jeżeli duże fragmenty kodu pochodzą z AI, za całą pracę uczeń otrzymuje ocenę niedostateczną (1) bez możliwości poprawy.**
- Sygnałem ostrzegawczym jest m.in. kod wykraczający daleko poza materiał z lekcji, którego uczeń nie potrafi objaśnić, styl kodu niepasujący do reszty projektu albo brak historii commitów pokazującej postęp pracy.

3.1 Galeria zdjęć z filtrowaniem

Strona z galerią co najmniej 9 zdjęć podzielonych na 3 kategorie (np. przyroda, miasto, zwierzęta).

- Miniatury zdjęć wyświetlane w siatce. Każde zdjęcie ma atrybut `data-kategoria`.
- Kliknięcie miniatury wyświetla to zdjęcie w dużym podglądzie nad galerią (zmiana atrybutu `src`) oraz jego podpis (z atrybutu `alt`).
- Aktualnie wybrana miniatura otrzymuje klasę `aktywne` (ramka), a pozostałe ją tracą.
- Przyciski „Wszystkie", „Przyroda", „Miasto", „Zwierzęta" pokazują tylko zdjęcia z wybranej kategorii (`show()` / `hide()`).
- Kliknięty przycisk filtra jest wyróżniony klasą CSS.
- Pod galerią licznik: „Wyświetlono: 3 z 9 zdjęć", aktualizowany przy każdej zmianie filtra.
- Przyciski „Poprzednie" i „Następne" przełączają zdjęcie w dużym podglądzie.

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)

3.2 Lista zakupów

Aplikacja do tworzenia listy zakupów.

- Pole tekstowe, pole ilości i przycisk „Dodaj". Dodawanie działa też po naciśnięciu Enter (formularz + `submit`).
- Walidacja: nazwa nie może być pusta ani składać się z samych spacji (`trim()`), ilość musi być liczbą większą od 0. Błędy wyświetlane na czerwono pod formularzem.
- Nowa pozycja dodawana do listy w formacie „Mleko – 2 szt." wraz z przyciskiem „Usuń".
- Kliknięcie nazwy produktu oznacza go jako kupiony (`toggleClass`: przekreślenie, szary kolor) i ponowne kliknięcie cofa oznaczenie.
- Licznik: „Do kupienia: 4, kupione: 2", aktualizowany po każdej zmianie.
- Przycisk „Usuń kupione" usuwa wszystkie oznaczone pozycje.
- Gdy lista jest pusta, wyświetla się komunikat „Lista jest pusta".

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)

3.3 Quiz wiedzy

Quiz z 8 pytaniami z wybranej dziedziny (np. informatyka, geografia), każde z 4 odpowiedziami w formie przycisków.

- Wyświetlane jest tylko jedno pytanie naraz (pozostałe ukryte), z numerem: „Pytanie 3 z 8".
- Poprawna odpowiedź oznaczona jest atrybutem `data-poprawna`.
- Po kliknięciu odpowiedzi: poprawna podświetla się na zielono, błędnie wybrana na czerwono, a wszystkie przyciski tego pytania zostają zablokowane (`prop("disabled", true)`).
- Przycisk „Dalej" pokazuje kolejne pytanie i jest aktywny dopiero po udzieleniu odpowiedzi.
- Bieżąca liczba punktów widoczna przez cały czas.
- Po ostatnim pytaniu ekran wyniku: liczba punktów, procent i ocena słowna (np. poniżej 50% „Spróbuj jeszcze raz").
- Przycisk „Zagraj ponownie" przywraca quiz do stanu początkowego.

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)

3.4 Kalkulator BMI z historią

Kalkulator wskaźnika BMI z zapamiętywaniem ostatnich wyników.

- Formularz: imię, waga (kg), wzrost (cm), przycisk „Oblicz”; obsługa przez `submit` bez przeładowania strony.
- Walidacja: imię niepuste, waga i wzrost muszą być liczbami (`Number()`, `isNaN()`), waga w zakresie 20–300, wzrost w zakresie 100–250. Każdy błąd ma osobny komunikat.
- Pole z błędem otrzymuje czerwoną ramkę (klasa CSS), która znika po poprawieniu.
- Wynik: BMI z dokładnością do 1 miejsca po przecinku (`toFixed(1)`) oraz kategoria (niedowaga, prawidłowa, nadwaga, otyłość), wyświetlona w kolorze odpowiadającym kategorii.
- Każdy wynik trafia na początek listy historii (`prepend`) w formacie „Ola – BMI 22.4 (prawidłowa)”.
- Przycisk „Wyczyść historię” usuwa wszystkie wpisy.

Minimalna liczba commitów: 7 (start projektu + 6 wymagań)

3.5 Generator wizytówki

Formularz, który na bieżąco tworzy podgląd wizytówki.

- Pola: imię i nazwisko, stanowisko, telefon, e-mail.
- Każde pole aktualizuje podgląd wizytówki na żywo (zdarzenie `input`). Puste pole wyświetla w podglądzie tekst zastępczy (np. „Imię i nazwisko”).
- Lista `select` z kolorem tła wizytówki (min. 4 kolory) oraz lista z rozmiarem czcionki (min. 3 rozmiary).
- Checkbox „Pokaż e-mail” pokazuje lub ukrywa e-mail na wizytówce.
- Checkbox „Tryb ciemny wizytówki” zmienia kolor tekstu na jasny lub ciemny.
- Przycisk „Zatwierdź”: sprawdza, czy imię i nazwisko oraz stanowisko są wypełnione, a telefon zawiera tylko cyfry (`isNaN`). W razie błędu wyświetla komunikat, a w razie sukcesu komunikat „Wizytówka gotowa!”.

Minimalna liczba commitów: 7 (start projektu + 6 wymagań)

3.6 Gra „Zgadnij liczbę”

Komputer losuje liczbę, a gracz próbuje ją odgadnąć.

- Lista `select` z poziomem trudności: łatwy (1–10), średni (1–50), trudny (1–100). Zmiana poziomu rozpoczyna nową grę.
- Losowanie liczby z zakresu poziomu (`Math.random()`, `Math.floor()`).
- Pole na liczbę i przycisk „Sprawdź”. Walidacja: pole niepuste, wartość jest liczbą i mieści się w zakresie poziomu.
- Podpowiedzi: „Za mało”, „Za dużo” albo „Brawo! Zgadłeś w 5 próbach”.
- Licznik prób oraz lista dotychczasowych prób (np. „25 – za dużo”).
- Po wygranej przycisk „Sprawdź” i pole zostają zablokowane.
- Przycisk „Nowa gra” losuje nową liczbę, czyści historię i odblokowuje pole.

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)

4.1 Lista zadań (To-Do) z priorytetami i filtrami

Menedżer zadań z podziałem na priorytety.

- Formularz: treść zadania, priorytet (`select`: niski, średni, wysoki), przycisk „Dodaj" (obsługa `submit`). Walidacja: treść niepusta i ma co najmniej 3 znaki.
- Każde zadanie wyświetla treść, priorytet (kolorowa etykieta zależna od priorytetu, nadana przez klasę CSS) oraz przyciski „Gotowe" i „Usuń".
- „Gotowe" przełącza stan zadania (`toggleClass`), a „Usuń" usuwa je po potwierdzeniu (`confirm()`).
- Filtry: „Wszystkie", „Aktywne", „Ukończone" oraz osobny filtr po priorytecie (`select`). Oba filtry działają jednocześnie.
- Edycja treści: kliknięcie przycisku „Edytuj" zamienia tekst zadania na pole tekstowe, a „Zapisz" zapisuje zmianę (z tą samą walidacją).
- Licznik: „Zadań: 7, ukończonych: 3 (43%)".
- Przyciski „Zaznacz wszystkie jako gotowe" i „Usuń ukończone".
- Gdy filtr nie pokazuje żadnego zadania, wyświetla się komunikat „Brak zadań do wyświetlenia".

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

4.2 Sklep z koszykiem

Prosty sklep internetowy z koszykiem i podsumowaniem zamówienia.

- Co najmniej 8 produktów zapisanych w HTML: zdjęcie, nazwa, cena; cena i nazwa w atrybutach `data-cena`, `data-nazwa`.
- Przycisk „Do koszyka" dodaje produkt do koszyka. Jeśli produkt już w nim jest, zwiększa się jego ilość zamiast dodania nowej pozycji.
- W koszyku przy każdej pozycji: nazwa, cena jednostkowa, przyciski „−" i „+", wartość pozycji i przycisk „Usuń". Zmniejszenie ilości do 0 usuwa pozycję.
- Suma koszyka przeliczana po każdej zmianie (pętla `each()` po pozycjach).
- Dostawa 15 zł, darmowa od 200 zł. Pod sumą komunikat: „Do darmowej dostawy brakuje 35 zł".
- Pole kodu rabatowego: „SZKOLA10" daje 10% zniżki, inny kod wyświetla komunikat o błędzie.
- Licznik produktów przy ikonie koszyka („Koszyk (5)") i komunikat „Koszyk jest pusty", gdy nie ma pozycji.
- Przycisk „Zamów" aktywny tylko, gdy koszyk nie jest pusty; po kliknięciu wyświetla podsumowanie i czyści koszyk.

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

4.3 Formularz rejestracyjny wieloetapowy

Rejestracja podzielona na 3 kroki z paskiem postępu.

- Krok 1: login (min. 4 znaki, bez spacji), e-mail (musi zawierać „@" i „."). Krok 2: hasło (min. 8 znaków, co najmniej jedna cyfra), powtórzenie hasła, wiek (liczba 13–120). Krok 3: zgoda na regulamin (checkbox), newsletter (checkbox), podsumowanie danych.
- Na ekranie widoczny jest tylko jeden krok (`show()` / `hide()`).
- Przycisk „Dalej" przechodzi do kolejnego kroku tylko wtedy, gdy wszystkie pola bieżącego kroku są poprawne. Przycisk „Wstecz" wraca bez utraty wpisanych danych.
- Komunikat błędu wyświetlany przy konkretnym polu, a pole z błędem otrzymuje klasę `blad`.
- Wskaźnik siły hasła aktualizowany na żywo (`input`): słabe, średnie, silne — z kolorem.
- Pasek postępu (zmiana szerokości przez `css()`) i napis „Krok 2 z 3".
- Krok 3 wyświetla podsumowanie wpisanych danych (hasło zamaskowane gwiazdkami). Przycisk „Zarejestruj" aktywny tylko po zaznaczeniu zgody.
- Po rejestracji formularz znika i pojawia się komunikat powitalny z loginem.

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

4.4 Kółko i krzyżyk

Gra dla dwóch graczy na jednym komputerze.

- Plansza 3×3 zbudowana z elementów HTML (np. `div`), stylowana w CSS.
- Gracze wykonują ruchy naprzemiennie. Kliknięcie wolnego pola wstawia X lub O; zajętego pola nie można nadpisać.
- Napis informuje, czyja jest tura („Ruch gracza: O").
- Wykrywanie wygranej we wszystkich 8 liniach (3 wiersze, 3 kolumny, 2 przekątne) oraz remisu.
- Wygrywająca linia zostaje podświetlona (klasa CSS), a plansza zablokowana do końca rundy.
- Pola na imiona graczy przed rozpoczęciem gry (z walidacją, że nie są puste) i tablica wyników serii: wygrane X, wygrane O, remisy.
- Przyciski „Nowa runda" (czyści planszę, zachowuje wyniki) i „Resetuj wyniki".
- Rundy rozpoczyna naprzemiennie X i O.

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

4.5 Konfigurator komputera

Konfigurator zestawu komputerowego z ceną liczoną na żywo.

- Listy `select` dla: procesora, karty graficznej, pamięci RAM i dysku (min. 3 opcje każda). Cena każdej opcji w atrybucie `data-cena`.
- Co najmniej 5 checkboxów z dodatkami (np. monitor, klawiatura, mysz) z cenami w `data-cena`.
- Cena całkowita aktualizowana automatycznie po każdej zmianie (`change`), z użyciem pętli `each()` po zaznaczonych dodatkach.
- Zmiana karty graficznej zmienia obrazek podglądu (zmiana atrybutu `src`).
- Lista podsumowania („Twój zestaw") budowana dynamicznie z nazw wybranych podzespołów i dodatków.
- Raty: `select` z liczbą rat (1, 6, 12, 24) i wyliczenie raty miesięcznej z dokładnością do 2 miejsc po przecinku.
- Ostrzeżenie, gdy cena przekroczy budżet wpisany przez użytkownika w osobnym polu (walidacja, że budżet jest liczbą).
- Przycisk „Resetuj konfigurację" przywraca ustawienia początkowe.

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

4.6 Notatnik z wyszukiwarką

Aplikacja do tworzenia i przeszukiwania notatek.

- Formularz: tytuł, treść (`textarea`), kategoria (`select`: szkoła, dom, praca), przycisk „Zapisz". Walidacja: tytuł i treść niepuste, tytuł maks. 40 znaków.
- Licznik znaków dla tytułu na żywo („12 / 40"), czerwony po przekroczeniu limitu; przycisk „Zapisz" zablokowany, gdy limit jest przekroczony.
- Notatki wyświetlane jako kafelki z kolorem zależnym od kategorii (klasa CSS) i przyciskami „Edytuj" i „Usuń".
- „Edytuj" wczytuje notatkę do formularza; ponowne zapisanie aktualizuje ją zamiast tworzyć nową.
- Wyszukiwarka na żywo (`input`): pokazuje tylko notatki, których tytuł lub treść zawiera wpisany tekst, bez rozróżniania wielkości liter.
- Filtr kategorii działający razem z wyszukiwarką.
- Licznik „Znaleziono: 3 z 10 notatek".
- Kliknięcie kafelka rozwija lub zwija pełną treść notatki (`toggle()`).

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

5.1 Gra Memory z poziomami trudności

Gra w odkrywanie par kart.

- Trzy poziomy trudności do wyboru (`select`): 12 kart (6 par), 16 kart (8 par), 20 kart (10 par). Plansza generowana w JavaScript pętlą (`for` + `append`), a nie wpisana ręcznie w HTML.
- Karty tasowane losowo przy każdej nowej grze.
- Kliknięcie odkrywa kartę. Po odkryciu dwóch kart: para zostaje odkryta na stałe, a niepasujące karty zakrywają się po 1 sekundzie (`setTimeout`). W tym czasie nie można klikać kolejnych kart.
- Licznik ruchów i stoper liczący czas gry (`setInterval`), uruchamiany przy pierwszym kliknięciu.
- Po odkryciu wszystkich par: komunikat z czasem i liczbą ruchów, stoper zatrzymany.
- Najlepszy wynik (najmniej ruchów) zapisywany osobno dla każdego poziomu w `localStorage` i wyświetlany nad planszą.
- Przycisk „Nowa gra" w dowolnym momencie.

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)

5.2 Gra zręcznościowa „Zbieracz"

Gra sterowana klawiaturą, w której postać zbiera punkty na planszy.

- Plansza o stałym rozmiarze i postać (element HTML) poruszająca się strzałkami lub klawiszami WASD (`keydown`, `event.key`, zmiana `left` / `top` przez `css()`). Postać nie może wyjść poza planszę.
- Na planszy w losowym miejscu pojawia się „moneta". Zebranie jej (wykrycie kolizji przez porównanie pozycji) dodaje punkt i losuje nową pozycję.
- Co 10 punktów pojawia się dodatkowa przeszkoda w losowym miejscu. Kontakt z przeszkodą kończy grę.
- Czas gry 60 sekund odliczany w dół (`setInterval`) i wyświetlany na ekranie.
- Ekran startowy z instrukcją i przyciskiem „Start", ekran końca gry z wynikiem i przyczyną zakończenia (czas lub przeszkoda).
- Rekord zapisywany w `localStorage` i wyświetlany na ekranie startowym.
- Pauza pod klawiszem „P" (zatrzymanie czasu i ruchu).

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)

5.3 Planer budżetu domowego

Aplikacja do śledzenia przychodów i wydatków.

- Formularz: opis, kwota, typ (przychód/wydatek), kategoria (`select`, min. 5 kategorii). Walidacja: opis niepusty, kwota jest liczbą większą od 0, maks. 2 miejsca po przecinku.
- Operacje przechowywane w tablicy obiektów i wyświetlane w tabeli (`append` wierszy `tr`): opis, kategoria, kwota (przychód na zielono, wydatek na czerwono), przycisk „Usuń".
- Podsumowanie: suma przychodów, suma wydatków, saldo (kolor zależny od znaku).
- Zestawienie wydatków według kategorii: kwota i udział procentowy, z prostym paskiem w CSS (szerokość proporcjonalna do udziału).
- Filtry: typ operacji i kategoria, działające łącznie. Sortowanie po kwocie rosnąco/malejąco.
- Edycja istniejącej operacji (wczytanie do formularza i zapis zmian).
- Wszystkie dane zapisywane w `localStorage` (`JSON.stringify` / `JSON.parse`) i odtwarzane po odświeżeniu strony.

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)

5.4 Tablica Kanban

Tablica do zarządzania zadaniami w trzech kolumnach: „Do zrobienia", „W trakcie", „Zrobione".

- Formularz dodawania zadania: tytuł, opis, priorytet, osoba odpowiedzialna. Walidacja wszystkich pól.
- Zadania wyświetlane jako karty z kolorem zależnym od priorytetu.
- Przyciski „←" i „→" na karcie przenoszą ją między kolumnami (w pierwszej kolumnie brak „←", w ostatniej brak „→").
- Limit 3 zadań w kolumnie „W trakcie": przy próbie przekroczenia wyświetla się ostrzeżenie, a karta nie zostaje przeniesiona.
- Licznik kart w nagłówku każdej kolumny.
- Edycja i usuwanie karty (z potwierdzeniem).
- Wyszukiwarka na żywo filtrująca karty we wszystkich kolumnach po tytule i osobie.
- Stan tablicy zapisywany w `localStorage` i odtwarzany po odświeżeniu strony.

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

5.5 Rezerwacja miejsc w kinie

System rezerwacji biletów na seans.

- Lista `select` z 3 seansami; każdy ma własną salę i własne zajęte miejsca.
- Sala 8 rzędów × 10 miejsc generowana pętlą `for` w JavaScript; rzędy oznaczone literami A–H, miejsca numerami.
- Ok. 20% miejsc losowo zajętych przy pierwszym wyborze seansu; zajęte miejsca nie są klikalne.
- Kliknięcie wolnego miejsca zaznacza je lub odznacza; maksymalnie 6 miejsc naraz (komunikat przy próbie przekroczenia).
- Ceny: rzędy A–F 25 zł, rzędy G–H (VIP) 35 zł. Dla każdego wybranego miejsca `select` typu biletu: normalny lub ulgowy (−30%).
- Podsumowanie na żywo: lista miejsc (np. „C5 – ulgowy – 17,50 zł") i suma.
- Formularz danych: imię, nazwisko, e-mail, telefon (9 cyfr) z walidacją. Po potwierdzeniu wybrane miejsca stają się zajęte, a na ekranie pojawia się potwierdzenie rezerwacji.
- Rezerwacje zapisywane w `localStorage` osobno dla każdego seansu.

Minimalna liczba commitów: 9 (start projektu + 8 wymagań)

5.6 Quiz na czas z rankingiem

Rozbudowany quiz z bazą pytań i tablicą najlepszych wyników.

- Baza co najmniej 20 pytań zapisana w tablicy obiektów JavaScript (treść, 4 odpowiedzi, indeks poprawnej). Pytania i odpowiedzi wyświetlane dynamicznie, nie wpisane w HTML.
- Każda gra losuje 10 różnych pytań (bez powtórzeń), a kolejność odpowiedzi jest tasowana.
- Na każde pytanie 15 sekund (`setInterval`) z paskiem czasu kurczącym się w CSS. Brak odpowiedzi w czasie liczy się jako błędna.
- Punktacja zależna od szybkości: 10 pkt bazowo + 1 pkt za każdą pozostałą sekundę.
- Po odpowiedzi podświetlenie poprawnej i błędnej odpowiedzi oraz automatyczne przejście dalej po 2 sekundach (`setTimeout`).
- Ekran końcowy: wynik, liczba poprawnych odpowiedzi oraz lista pytań, na które odpowiedziano błędnie, z poprawnymi odpowiedziami.
- Przed grą pole na imię (walidacja). Ranking 5 najlepszych wyników zapisywany w `localStorage` i wyświetlany na ekranie startowym.

Minimalna liczba commitów: 8 (start projektu + 7 wymagań)